File Permissions in Linux (chmod, chown)

Introduction

Imagine you are working on a **Linux system**, and you need to:

- **Make a file readable only by you.**
- **Allow team members to edit a shared file.**
- **Change file ownership to another user.**

How do you control who can access or modify files?

This is where chmod and chown come in!

Section 1: Learn

What Are File Permissions?

File permissions in Linux control who can read, write, and execute a file.

They help:

1. **Protect files from unauthorized access.**
2. **Ensure only authorized users can modify critical files.**
3. **Manage multi-user systems securely.**

Understanding File Permissions

Run the following command to check file permissions:

```
ls -l
```

Example Output:

```
-rwxr-xr-- 1 user group 1234 Feb 26 10:00 myfile.txt
```

Breaking Down the Output:

Symbol	Meaning
-	Regular file (d means directory)
rwx	Owner has **read (r), write (w), execute (x)** permission
r-x	Group has **read (r) and execute (x)** permission
r--	Others have **read (r) only** permission
user	Owner of the file
group	Group associated with the file
1234	File size in bytes
Feb 26 10:00	Last modified date
myfile.txt	File name

Why Do We Need File Permissions?

Without File Permissions	**With File Permissions**
Any user can modify any file	Restricts access based on user roles
Risk of accidental deletions	Prevents unauthorized modifications
No security on shared systems	Ensures **only authorized users** can edit files
Difficult to manage multi-user environments	Allows structured file ownership and access control

How Do File Permissions Work?

Each file has three permission groups:

1. **Owner (u)** → The user who created the file.
2. **Group (g)** → Users in the same group as the owner.
3. **Others (o)** → Everyone else on the system.

An Interesting Anecdote

In early Unix systems, **any user could modify any file**, leading to **security risks**. File permissions were introduced to **protect sensitive files**, and today, Linux systems use chmod and chown to enforce security.

Section 2: Practice

Step-by-Step Guide to Using chmod and chown

1. Changing File Permissions (chmod)

```
# Give full permissions (read, write, execute) to the owner only
chmod 700 myfile.txt

# Make a file readable and writable by everyone
chmod 666 myfile.txt

# Allow only the owner and group to modify the file
chmod 770 myfile.txt

# Remove write permission from others
chmod o-w myfile.txt
```

How does this help?

- Protects sensitive files **from unauthorized access**.

- Ensures **team members can edit shared files securely**.

---

2. Changing File Ownership (chown)

```
# Change file owner to "newuser"
sudo chown newuser myfile.txt

# Change both owner and group
sudo chown newuser:newgroup myfile.txt

# Transfer ownership of a directory and its files
sudo chown -R newuser:newgroup myfolder/
```

Why is this important?

- Allows **transferring ownership of files between users**.
- Helps system administrators **manage permissions in teams**.

---

3. Understanding Numeric (Octal) Permissions

File permissions are represented as a **3-digit number (rwx → 4+2+1)**:

Permission	Symbolic	Numeric
Read	r--	4
Write	-w-	2
Execute	--x	1
No permission	---	0

Example:

```
chmod 755 myscript.sh
```

User	Permissions	Value
Owner	rwx (read, write, execute)	7
Group	r-x (read, execute)	5
Others	r-x (read, execute)	5

How does this help?

- Ensures **scripts can be executed by users**.
- Prevents **accidental modifications by others**.

Real-World Example

A **web developer** is deploying a website on a **Linux server**. They need to:

1. **Allow the web server to read files but not modify them.**
2. **Ensure only developers can edit website files.**
3. **Prevent unauthorized users from accessing sensitive configurations.**

What did they do?

1. Used chmod 644 to **set correct file permissions**.
2. Used chown www-data:webadmins to **set ownership**.
3. Configured **directories to prevent accidental deletions**.

Result? A **secure, organized, and well-maintained** website.

Section 3: Know More

Frequently Asked Questions

1. **How do I check file permissions?**

```
ls -l filename
```

2. **What does** chmod 777 **do?**
    - **Everyone can read, write, and execute** the file.
    - **Risky for sensitive files** (use carefully).
3. **How do I make a script executable?**

```
chmod +x script.sh
```

4. This allows the script **to be run as a program**.
5. **What is the difference between** chmod **and** chown**?**
    - chmod **modifies file permissions** (who can read/write/execute).
    - chown **changes file ownership** (who owns the file).
6. **Where can I practice Linux file permissions?**
    - Use a **Virtual Machine (VM) with Ubuntu**.
    - Try online Linux terminals like **JSLinux or Webminal**.

Conclusion

Linux file permissions (chmod, chown) **help protect data, control access, and manage system security**.

Start by **checking file permissions (**ls -l**), experiment with** chmod **and** chown**, and soon you'll be handling Linux security like an expert!**